

Enhancing Retrieval-Augmented Generation for Text Completion Through Query Selection

Idan Pogrebinsky
idan.pogy@campus.technion.ac.il
Technion
Haifa, Israel

David Carmel
david.carmel@tii.ae
TII and Technion
Haifa, Israel

Oren Kurland
kurland@technion.ac.il
Technion
Haifa, Israel

Abstract

In a Retrieval-Augmented Generation (RAG) system designed for the fundamental task of text completion, a query is derived from the prompt of a large language model (LLM) and is issued to an external resource. The retrieved content is then incorporated into the prompt to enhance text completion. Since prompts can be considerably long, it is common practice to use a short suffix of the prompt as the query. We empirically show, using a suite of oracle experiments, that this approach is often suboptimal with respect to other choices of a query from the prompt. This finding gives rise to a novel research challenge: identifying the optimal query for RAG in this setting. As an initial study, we propose a few query selection methods, some of which yield statistically significant improvements over using the prompt's suffix as a query.

CCS Concepts

• Information systems → Information retrieval.

Keywords

LLM, RAG, Text completion, Query Selection

ACM Reference Format:

Idan Pogrebinsky, David Carmel, and Oren Kurland. 2025. Enhancing Retrieval-Augmented Generation for Text Completion Through Query Selection. In *Proceedings of the 2025 International ACM SIGIR Conference on Innovative Concepts and Theories in Information Retrieval (ICTIR '25)*, July 18, 2025, Padua, Italy. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3731120.3744610>

1 Introduction

Retrieval augmented generation (RAG) enhances large language models (LLMs) generation performance by augmenting the LLM input (prompt) with content extracted from external corpora [5, 9, 12]. In this work we focus on the in-context RAG-LLM setting [8, 20] for the fundamental task of next-token generation. In this setting we assume a frozen LLM (the generator) and a frozen information retrieval (IR) component (the retriever). At each generation stage, the retriever extracts content from external corpora, to be incorporated into the original prompt for generation enhancement.

Previous studies [8, 20] clearly showed that RAG significantly improves the LLM's generation performance in the in-context RAG-LLM setting. Ram et al. [20] selected a short suffix of the prompt as

a query, assuming that the prompt's suffix is the most influential content on next-token generation; hence, content relevant to this suffix might be helpful for the generation task. Recently, Huly et al. [8] found empirically that a query composed of the last 32 tokens of the prompt performs the best for the standard Wikitext-103 benchmark [18] when using several retrieval approaches and LLMs.

In this paper, we empirically show, using a suite of oracle experiments, that using the prompt suffix as a query is often suboptimal with respect to other query selection methods. More specifically, we show that the suffix-based query has a significant number of sub-queries that outperform it, improving the resultant perplexity performance sometimes by more than 40%. These findings give rise to a novel research challenge: *identifying the optimal sub-query of a prompt suffix for RAG used for the text-completion task*. More generally, ranking sub-queries by the presumed resultant perplexity of using them for RAG is an emerging novel challenge.

We also present an initial study of several query selection methods, all are based on predicting the resultant perplexity of using the selected query for RAG. More specifically, we rank the sub-queries using the predicted values and select the highest ranked for RAG. We show that some of our query selection methods perform statistically significantly better than using the suffix-based query, for three different LLMs. The best performing methods are (i) a BERT-based model trained to predict the contribution of a retrieved passage for next-token generation and (ii) estimates based on the entropy of the next-token distributions induced by the LLM for the prompt.

Our contributions can be summarized as follows:

- Showing using oracle experiments that the practice of using the suffix of the prompt as a query in RAG for text completion can be sub-optimal with respect to a significant number of sub-queries of this suffix. This finding gives rise to novel research questions: identifying the optimal sub-query and, to this end, ranking sub-queries by the presumed resultant perplexity.
- Initial study of using several methods to select sub-queries by ranking them. Some of the methods result in perplexity that is statistically significantly better than that of using the full query (i.e., the prompt suffix).

2 Related Work

Query reformulation for RAG systems has mostly been studied in the context of improving RAG performance for knowledge intensive tasks such as question answering [16, 17, 19, 26, 27] and conversations [4, 14]. These approaches encompass question decomposition (e.g., asking multi-hop questions [19]), query rewriting [16, 17, 26],



This work is licensed under a Creative Commons Attribution 4.0 International License. *ICTIR '25, Padua, Italy*

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1861-8/2025/07

<https://doi.org/10.1145/3731120.3744610>

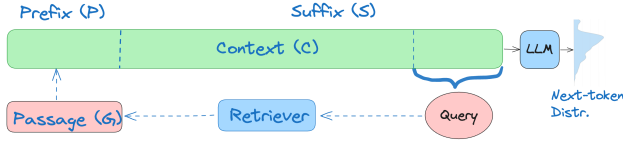


Figure 1: The in-context RAG-LLM setting [7, 8, 20]. (The figure is taken from Huly et al. [7].) The text that results from detokenizing the last tokens is used to query the Retriever. The highest ranked passage replaces the prefix of the context.

and query augmentation that supplements the input with additional information [2, 22, 27]. A common approach applied in these methods is the utilization of an auxiliary LLM to supplement the question with additional information, thereby substantially facilitating the retrieval process. As a representative example, Ma et al. [16] experimented with a query rewriting approach where a small language model (T5) is adopted as a trainable query rewriter, fine tuned with reinforcement learning based on the generator feedback, to adapt the query for improving downstream task.

Another area of related research is query reduction for verbose queries in ad hoc retrieval [6]: relatively long queries are reduced to shorter ones which traditional information retrieval techniques can handle better. To the best of our knowledge, no previous work has directly studied query reformulation or query reduction in the context of RAG-based text completion.

3 Query Selection Models

3.1 Background: In-context RAG

We subscribe to the in-context RAG-LLM framework for text completion [7, 8, 20] depicted in Figure 1. We assume the context is of a fixed length, determined by the LLM’s input capacity. We split it to a prefix, the leftmost tokens in the context, and a suffix. A query is derived from the context and is used by the Retriever to retrieve a ranked passage list from a corpus. The top ranked passages are used to replace the prefix.

Following Ram et al. [20], the query used as a baseline is the text that results from detokenizing the rightmost tokens in the context. Ram et al. [20] also demonstrated in their setting that using more than one passage for text completion has no merit. We therefore augment the context with the top-1 passage in our setting. Finally, the prefix length is set to match the replacing passage length. The resultant modified context, namely the passage followed by the suffix, is used as a prompt to the LLM to induce a RAG-based next-token distribution over the token space.

3.2 Query Selection

As we show in Section 4, using the rightmost tokens of the prompt, henceforth referred to as **full query**, to query the external corpus is often suboptimal with respect to using other queries extracted

Table 1: An example of a sub-query outperforming the full suffix-based query. The content retrieved for the sub-query of $k=5$ words is more useful for predicting “white”, the actual next token, than the content retrieved for the full query of $k=25$ words.

Full query	Oracle subquery ($k = 5$)	Next token
...men had killed survivors on the ground. The majority of Rhodesians, both black and white, saw the attack as an act of terrorism. A fierce ____	Rhodesians attack act terrorism fierce	white

from the prompt. Table 1 shows an example where a sub-query extracted from the suffix – using an oracle knowledge of the resultant perplexity – significantly outperforms the full query.

We therefore present a few methods to select a query from a prompt. Specifically, our goal is to extract the optimal sub-query from the prompt’s suffix¹. The optimal sub-query is the one whose corresponding *RAG-prompt* leads to the minimal perplexity of the next token distribution with respect to all other sub-queries.

Our query selection methods are based on extracting N sub-queries of length k , where the k words are randomly selected out of the l rightmost words of the prompt. (Herein l is the length of the full, original, query in words.). Unlike previous work, we extract words rather than tokens (sub-words) from the suffix, as arbitrary k selected tokens are not necessarily de-tokenized to a coherent word-based query. We predict the perplexity of the next-token distribution induced using the *RAG-prompt* for each sub-query², and select the sub-query for which the predicted perplexity is the lowest. Since our goal is ranking the sub-queries by the predicted values, some of the predictors we use assign a value which is not a direct estimate for perplexity, but is rather assumed to be correlated with perplexity.

Efficiency Considerations. An exhaustive search over the entire space of $\binom{l}{k}$ sub-queries is computationally difficult. Therefore, we use a random sample of N sub-queries of length k ($N \ll \binom{l}{k}$), with the understanding that the best sub-query in the sample may not necessarily be the optimal one; however, its resultant perplexity is an upper bound on the minimal perplexity that can be attained by query selection. Nevertheless, we show below that this approach still results in perplexity that is statistically significantly lower than that of using the original full query.

In addition, we note that executing N searches for all sub-queries in the sample to obtain their top-ranked passage, is also computationally demanding. Fortunately, we use sparse retrieval. Hence, the N searches can be executed jointly with the search for the full query. By one traversal over the index, we can identify the top passage for the full query (of length l), and the top passage for each of the N sub-queries (of length k). Thus, only one retrieval call is needed per each generation task.

¹Our search for optimal query is limited to the prompt’s suffix, based on the In-Context RAG-LLM assumption that the major impact on next-token generation comes from the prompt’s rightmost tokens [20].

²For brevity, we often write “the (resultant) perplexity of the sub-query” instead of “the perplexity resulting from using the corresponding *RAG-prompt*”.

3.3 Predictors

We next describe several predictors which estimate the resultant perplexity for a given (sub-)query, or assign prediction values assumed to be correlated with the perplexity. The predictors can utilize the query, its retrieved passage, external corpus statistics, the original prompt and the induced next-token distribution.

3.3.1 Classical predictors. We first study several classical (unsupervised) query performance predictors (QPP) that were designed to estimate ad hoc document retrieval effectiveness without relevance judgments [3].

- **MAX_IDF:** A pre-retrieval predictor that uses the maximum IDF value of the query terms [11].
- **WIG:** A post-retrieval predictor that measures the difference between the average retrieval score of the top- n passages and the corpus retrieval score [29].
- **NQC:** A post-retrieval predictor that measures the standard deviation of retrieval scores of the top- n passages normalized by the corpus retrieval score [23].
- **Centrality** A post-retrieval predictor (cf., [3, 28]) that measures the similarity of a query's retrieved list to the retrieved lists of the other queries extracted from the same prompt's suffix. High similarity attests to the centrality of the query with respect to other queries. We use the average Rank-Biased Overlap (RBO) [24] ($p=0.9$), between the query's top- n retrieved list and the other queries' top- n lists, for an inter-list similarity measure.

All the predictors just mentioned estimate retrieval effectiveness based on presumed passage relevance to the query, while our goal is to estimate resultant perplexity. The predictors we present below use estimates that are more directly connected to perplexity.

3.3.2 Supervised prediction. We first tried BERT-QPP [1] for sub-query performance prediction. BERT-QPP is a supervised predictor that models the textual association of a query with top-retrieved documents. This model failed on this task since it is trained to estimate the document relevance to the query, while our goal is predicting its usefulness for text completion.

We therefore train a BERT-based model to estimate perplexity. *PredictiveReranker* is based on the ranking model described in [20]. This is a RoBERTa-based model [15]³ which takes as input the sub-query's retrieved passage followed by the suffix — truncated from left to fit RoBERTa's 512 token input limit — and returns a prediction score.

3.3.3 Next-token distribution analysis. Since perplexity is directly based on the next-token distribution induced by the LLM, we use properties of this distribution as predictors [10, 25]. For each *RAG-prompt* associated with a sub-query, we measure the entropy of the next-token distribution induced by the LLM. The premise is that high entropy reflects low confidence in next token generation, hence high perplexity, and vice versa.

We study three entropy-based predictors:

- **Entropy:** The entropy of the next-token distribution, induced for the sub-query's *RAG-prompt*.

³<https://huggingface.co/FacebookAI/roberta-base>

Table 2: Percentage of generation tasks having at least M dominating sub-queries.

Model	Sub-query length (k)	$M \geq 1$	$M \geq 25$	$M \geq 50$	$M \geq 75$
Falcon2-11B	6	89.54%	63.15%	44.75%	25.27%
	12	88.35%	60.47%	34.96%	12.76%
	18	83.16%	47.92%	21.50%	4.25%
Llama3.1-8B	6	86.23%	61.27%	43.69%	24.48%
	12	85.87%	57.74%	33.81%	13.29%
	18	80.87%	46.19%	21.79%	4.33%
GPT2-Small	6	89.19%	59.51%	41.72%	24.85%
	12	87.41%	57.07%	35.08%	12.72%
	18	80.97%	47.05%	22.15%	3.57%

- **AvgBackEntropy, VarBackEntropy:** High confidence during the LLM's auto-regressive generation process may suggest high confidence (thus low perplexity) for the next token generation as well. Therefore, we measure the entropy's average and variance over the sequence of next-token distributions, induced by the LLM across the last v tokens of the *RAG-prompt*. Formally, given a prompt $t_1, \dots, t_n$, we measure the entropy average (*AvgBackEntropy*) and variance (*VarBackEntropy*) for the last v induced distributions while *RAG-prompting* the LLM with $t_1, \dots, t_k$, where $k \in [n - v + 1, \dots, n]$.
- **AvgBackPPL, VarBackPPL:** Looking backward in the prompt suffix, the actual tokens are already known. Thus, we also measure the perplexity's average and variance based on the next-token distributions induced by the LLM across the last v tokens of the *RAG-prompt*.

4 Evaluation

4.1 Experimental Setup

Our experimental setup is based on the In-Context RAG-LLM framework [7, 8, 20]. In this framework, the external source is the English Wikipedia dump from Dec. 20, 2018, which contains $\sim 5.7M$ articles split into 100-word disjoint passages⁴. The retrieval system is based on Pyserini [13]. Previous work in this setting showed that sparse retrieval, including Okapi BM25 [21], substantially outperforms dense retrieval for the text completion task [8, 20]. Thus, we use Okapi BM25 as the retriever with default parameter values of Pyserini.

We use three LLMs: (i) GPT2-Small⁵, as a reference to previous studies [8, 20]; (ii) Llama3.1-8B⁶; and (iii) Falcon2-11B⁷, both are recent representatives of a high quality moderate-sized LLMs. We use a fixed prompt length, $L = 1024$, which is GPT2-small's maximal input length, following [7, 8, 20]. The suffix-based (full) query length, l , is set to 25, following [8] who showed that using the last 32 tokens of the prompt as a query is optimal in the In-Context RAG-LLM setting. Since 32 GPT2 tokens are, on average, 25 words, we set the suffix-based full query length l to 25, and study several sub-query lengths, relatively short to long with respect to the entire suffix. We sample $N = 100$ sub-queries of length k from the full query, $k \in \{6, 12, 18\}$, and rank them using each of the predictors.

⁴https://huggingface.co/datasets/facebook/wiki_dpr

⁵<https://huggingface.co/gpt2>

⁶<https://huggingface.co/meta-llama/Llama-3.1-8B>

⁷<https://huggingface.co/tiiuae/falcon-11B>

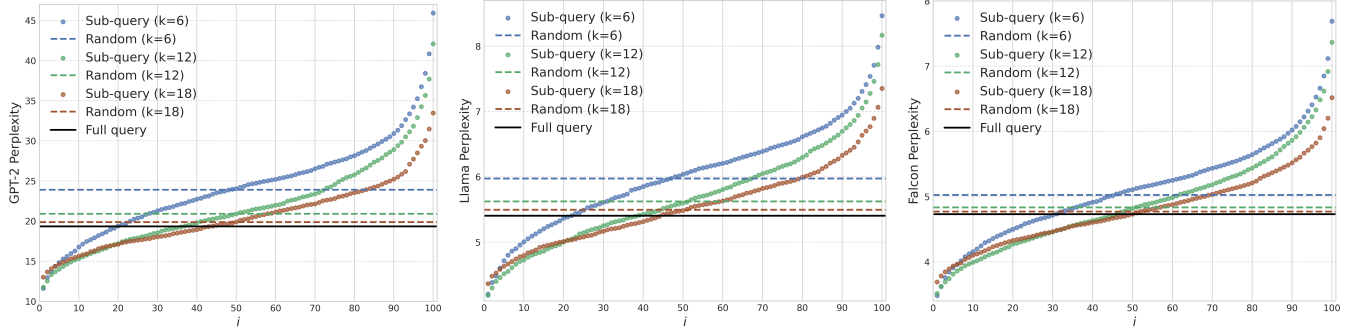


Figure 2: Average perplexity of the k -length sub-queries at position i in the rank, for GPT2 (Left), Llama-3.1-8B (Middle), Falcon2-11B (Right). The perplexity of randomly selected k -length sub-queries and of the full query serves for reference.

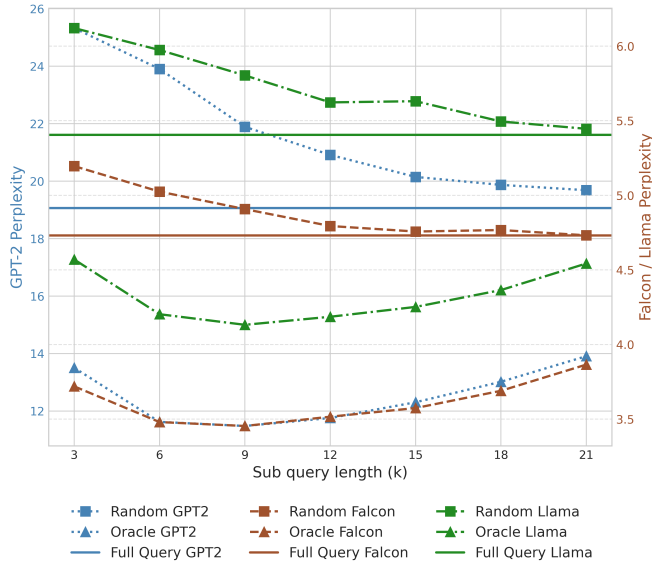


Figure 3: Perplexity as a function of sub-query length for two sub-query selection approaches: Random and Oracle.

We follow standard practice in work on evaluating language models and measure the perplexity of the next token distributions, induced by the LLM, with respect to the actual next tokens in the text. Perplexity is measured over the Wikitext-103 dataset [18] as follows. We randomly sample a **base set** of 4% of the generation tasks from the validation segment of Wikitext-103 (cf., [8]). Since each LLM uses its own tokenizer, the number of generation tasks in the base set is different for each LLM: 2406, 2520 and 2869 tasks for GPT2-Small, Llama3.1-8B, and Falcon2-11B, respectively. To have robust evaluation, and to allow for statistical significance testing, we randomly sample 500 generation tasks from the base set for 500 times. We compute the perplexity of a prediction method for each of the 500 samples, and report the resultant average perplexity over the 500 samples. We use a paired two-tailed t-test with $p \leq 0.05$ over the 500 samples to determine statistically significant differences.

We train the *PredictiveReranker* predictor following the procedure described in Ram et al. [20]. We randomly sample 300K different prompts, extracted from the train segment of Wikitext-103, and generate a *RAG-prompt* for each of the top-16 passages retrieved for their corresponding suffix-based query. Following Ram et al. [20], the prompt’s label is determined by the perplexity, computed by GPT2-small over a stride of the 4 actual following tokens in the text. The loss function follows previous work on RAG training [12]. For more details on the training process see Ram et al. [20].

For all classical post-retrieval predictors we set the number of retrieved passages, n , to 100. The backward window length, v , in the entropy-based and PPL predictors, is fixed to 16 as it demonstrates strong performance across all evaluated LLMs, as determined through a comprehensive grid search over the set $\{1, 2, 4, 8, 16, 32, 64\}$.

4.2 Oracle Experiments

We first study the potential of query selection by measuring the percentage of next-token generation tasks (out of all generation tasks in the base set) where at least $M \geq 1$ of the sub-queries (out of the 100 sampled) yield next-token perplexity that is lower than that of using the full query; these sub-queries are referred to as *dominating sub-queries*. This is an oracle setting where the actual next token for each generation task, and hence the actual perplexity, are known. Table 2 presents the results for the three LLMs and three sub-query lengths (k): 6, 12 and 18.

We see in Table 2 that for all LLMs, and for all sub-query lengths, for 80+% of the tasks there exists at least one sub-query dominating the full query, for 45+% of the tasks at least 25 dominating sub-queries, and for 21+% of the tasks at least 50 dominating sub-queries. Another interesting insight based on Table 2 is that there are more dominant short sub-queries than long sub-queries. This finding can potentially be attributed to the fact that for long sub-queries the similarity between the retrieved list and that retrieved for the full query is likely to be much higher than that for short sub-queries. Hence, there is more potential to find short sub-queries which are more effective than the full query than long sub-queries.

For the next oracle experiment, for each LLM we rank the 100 sub-queries (of length k) in ascending order of the resultant perplexity of using their corresponding *RAG-prompt*. Figure 2 shows the average

Table 3: Perplexity attained by using the predictors to select a sub-query. The perplexity of Oracle, Random, Full Query and NO-RAG are given for reference. All results are statistically significantly different from those of Full Query, PredictiveReranker and AvgBackEntropy, excluding those marked by 'f', 'a' and 'p', respectively. Boldface: the best result in a column.

Model / k	GPT2-Small			Llama3.1-8B			Falcon2-11B		
	6	12	18	6	12	18	6	12	18
Max IDF	23.072	20.838	20.111	5.858	5.598	5.474	4.978	4.774	4.771
WIG	24.275	21.439	20.520	6.029	5.764	5.539	5.142	4.914	4.791
NQC	23.589	20.651	20.1312	5.982	5.617	5.528	5.059	4.793	4.731 ^f
Centrality	20.552	19.651	19.393	5.586	5.446	5.417	4.805	4.738	4.732 ^f
Entropy	20.690	20.080	19.468	5.496	5.370	5.373	4.898	4.838	4.748
AvgBackEntropy	19.244	18.974	19.235	5.323	5.243	5.270	4.664	4.646	4.649
VarBackEntropy	21.298	20.825	20.6505	5.455	5.410 ^f	5.395 ^f	4.704	4.679	4.678
AvgBackPPL	18.977	18.781	18.896	5.4335	5.3378	5.3366	4.712	4.666	4.697
VarBackPPL	20.606	20.751	20.108	5.5263	5.3844	5.4184	4.801	4.812	4.772
PredictiveReranker	18.864	18.345	18.628	5.399 ^f	5.353	5.388	4.721 ^f	4.616	4.607
Oracle	11.621	11.756	13.017	4.203	4.186	4.365	3.480	3.516	3.689
Random	23.903	20.913	19.868	5.974	5.623	5.495	5.025	4.833	4.770
Full Query		19.339			5.405			4.731	
NO-RAG		24.574			6.098			5.111	

perplexity for sub-queries at position i in the ranking; $i \in [0, \dots, 99]$. The average perplexity of using the RAG-prompt for a randomly selected sub-query and for the full query is presented for reference.

We can see in Figure 2 that for all LLMs, the average perplexity for sub-queries of length $k = 18$, at position $i \leq 40$ in the ranking, is better (lower) than the average perplexity of the full query (horizontal solid black line). Consequently, on average, any query selection method that is able to select a sub-query, out of the top-40 sub-queries of length 18, is guaranteed to outperform the full query.

We also see in Figure 2 that there is a very small gap between the full query and a randomly selected sub-query, for sub-queries with $k = 12$ words and more. On the other hand, shorter randomly selected sub-queries are much worse than the full query. This suggests a very simple yet effective query selection method: randomly sample $k > 12$ terms from the suffix to guarantee (on average) performance similar to that of the full suffix-based query.

Lastly, Figure 3 depicts the perplexity of two sub-query selection approaches. *Random*: Randomly selecting a sub-query of a given length, and *Oracle*: selecting the sub-query that yields the lowest perplexity. As can be seen, *Random*'s performance improves with sub-query length, converging to the full query performance. The reason is that a small set of randomly selected words is unlikely to be useful while longer sub-queries are more similar to the full query; hence, they are more coherent and are more likely to bring value.

Figure 3 also shows that for all LLMs, *Oracle* substantially outperforms the full query, with an optimal performance for 9-words sub-query (40%, 29%, and 27% perplexity improvement for GPT2, Llama-3.1, and Falcon2, respectively). All in all, the oracle-based findings attest to the high potential of query selection for In-Context RAG-LLMs.

4.3 Query Selection Using the Predictors

We now turn to study the resultant perplexity of using the predictors from Section 3. Recall that each predictor is used to rank the 100 sub-queries sampled from the full query. The highest ranked sub-query is used to retrieve a passage. The perplexity resulting from using the RAG-prompt which includes the retrieved passage is reported. (As described in Section 3, the reported perplexity is the average over 500 samples of 500 generation tasks each.) For reference comparison, we report the perplexity attained by using the original prompt (NO-RAG), the fixed-length suffix-based full query (Full Query), the best sub-query among the 100 (Oracle), and a randomly selected sub-query (Random). Table 3 presents the results for the three LLMs and three lengths (k) of sub-queries.

Table 3 shows, as in previous reports [8, 20], that RAG substantially outperforms NO-RAG. Interestingly, even Random outperforms NO-RAG. We also see that most unsupervised classical predictors fail to select a good sub-query for all LLMs and all sub-query lengths: they are outperformed by the full query; a few of them are even outperformed by Random. This finding attests to the major gap between passage relevance to a query, which these predictors target, and the resultant perplexity of using the passage in the RAG-prompt.

Table 3 shows that the entropy-based predictors are quite effective: they statistically significantly outperform the full query in many cases. *AvgBackEntropy* statistically significantly outperforms the full query in all cases (3 sub-query lengths and 3 LLMs). The perplexity-based (PPL) predictors always statistically significantly outperform the full query. Yet, they are less effective than *AvgBackEntropy*. We also see that the variance-based PPL and Entropy predictors are less effective than the average-based counterparts.

We see in Table 3 that the supervised predictor, *PredictiveReranker*, always outperforms the full query, in most cases to a statistically significant degree. Its performance is never statistically significantly worse than that of the entropy and PPL predictors although it does not analyze next-token distributions.

Table 3 also shows that while the most effective predictors statistically significantly outperform Random, they are substantially outperformed by oracle selection of the best sub-query. Finally, we note the performance differences for the LLMs. As can be expected, larger modern models perform much better than GPT2-small. However, from a query selection perspective, we observe consistent relative performance patterns of predictors across the three LLMs.

5 Conclusions and Future Work

We demonstrated empirically, using oracle experiments, the huge potential of query selection for text completion in the In-Context RAG-LLM setting, with respect to using the original (full) suffix-based query commonly used in prior art. As an initial study, we presented the clear merits of using a few suggested predictors to rank sub-queries sampled from the full query and to select the highest ranked one for RAG. However, the resultant perplexity still lags far behind that of the optimal sub-query. This finding gives rise to a novel research challenge: finding an optimal query for In-Context RAG-LLM. We leave for future work the study of other query selection mechanisms such as optimizing the search process in the combinatorial sub-query space, or cascade ranking where

an initial retrieval is performed over all sub-queries using a light ranker; then, the top ranked sub-queries are re-ranked using a more elaborated ranker. Another complementary work left for the future is improving the prediction of RAG performance in this setting [7] for a better guidance of the query selection process.

5.1 Acknowledgments

We thank the reviewers for their comments. This work is supported in part by the Israel Science Foundation (grant no. 403/22).

References

- [1] Negar Arabzadeh, Maryam Khodabakhsh, and Ebrahim Bagheri. 2021. BERT-QPP: Contextualized Pre-trained transformers for Query Performance Prediction. In *Proceedings of CIKM*. 2857–2861.
- [2] Hiteshwar Kumar Azad and Akshay Deepak. 2019. Query expansion techniques for information retrieval: a survey. *Information Processing & Management* 56, 5 (2019), 1698–1735.
- [3] D. Carmel and E. Yom-Tov. 2010. *Estimating the Query Difficulty for Information Retrieval*. Morgan & Claypool Publishers.
- [4] Ahmed Elgohary, Denis Peskov, and Jordan Boyd-Graber. 2019. Can You Unpack That? Learning to Rewrite Questions-in-Context. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. 5918–5924.
- [5] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. 2024. Retrieval-Augmented Generation for Large Language Models: A Survey. *arXiv:2312.10997* [cs.CL]
- [6] Manish Gupta and Michael Bendersky. 2015. Information Retrieval with Verbose Queries. In *Proceedings of the 38th International ACM SIGIR Conference on Research and Development in Information Retrieval (Santiago, Chile) (SIGIR '15)*. Association for Computing Machinery, New York, NY, USA, 1121–1124. <https://doi.org/10.1145/2766462.2767877>
- [7] Oz Huly, David Carmel, and Oren Kurland. 2025. Predicting RAG Performance for Text Completion. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*. Association for Computing Machinery.
- [8] Oz Huly, Idan Pogrebinsky, David Carmel, Oren Kurland, and Yoelle Maarek. 2024. Old IR Methods Meet RAG. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2559–2563.
- [9] Gautier Izacard, Patrick Lewis, Maria Lomeli, Lucas Hosseini, Fabio Petroni, Timo Schick, Jane Dwivedi-Yu, Armand Joulin, Sebastian Riedel, and Edouard Grave. 2023. Atlas: Few-shot learning with retrieval augmented language models. *Journal of Machine Learning Research* 24, 251 (2023), 1–43.
- [10] Youna Kim, Hyuhng Joon Kim, Cheonbok Park, Choonghyun Park, Hyunsoo Cho, Junyeob Kim, Kang Min Yoo, Sang-goo Lee, and Taeuk Kim. 2024. Adaptive Contrastive Decoding in Retrieval-Augmented Generation for Handling Noisy Contexts. In *Findings of the Association for Computational Linguistics: EMNLP*. 2421–2431.
- [11] K. L. Kwok. 1996. A new method of weighting query terms for ad-hoc retrieval. In *Proceedings of SIGIR*. 187–195.
- [12] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems* 33 (2020), 9459–9474.
- [13] Jimmy Lin, Xueguang Ma, Sheng-Chieh Lin, Jheng-Hong Yang, Ronak Pradeep, and Rodrigo Nogueira. 2021. Pyserini: A Python Toolkit for Reproducible Information Retrieval Research with Sparse and Dense Representations. In *Proceedings of SIGIR*. 2356–2362.
- [14] Sheng-Chieh Lin, Jheng-Hong Yang, Rodrigo Nogueira, Ming-Feng Tsai, Chuan-Ju Wang, and Jimmy Lin. 2021. Multi-Stage Conversational Passage Retrieval: An Approach to Fusing Term Importance Estimation and Neural Query Rewriting. *ACM Trans. Inf. Syst.* 39, 4, Article 48 (Sept. 2021), 29 pages. <https://doi.org/10.1145/3446426>
- [15] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. RoBERTa: A Robustly Optimized BERT Pretraining Approach. *arXiv:1907.11692* [cs.CL] <https://arxiv.org/abs/1907.11692>
- [16] Xinbei Ma, Yeyun Gong, Pengcheng He, Nan Duan, et al. 2023. Query Rewriting in Retrieval-Augmented Large Language Models. In *The 2023 Conference on Empirical Methods in Natural Language Processing*.
- [17] Shengyu Mao, Yong Jiang, Boli Chen, Xiao Li, Peng Wang, Xinyu Wang, Pengjun Xie, Fei Huang, Huajun Chen, and Ningyu Zhang. 2024. RaFe: Ranking Feedback Improves Query Rewriting for RAG. *arXiv preprint arXiv:2405.14431* (2024).
- [18] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2016. Pointer Sentinel Mixture Models. In *Proceedings of the International Conference on Learning Representations (ICLR)*.
- [19] Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A Smith, and Mike Lewis. 2023. Measuring and Narrowing the Compositionality Gap in Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. 5687–5711.
- [20] Ori Ram, Yoav Levine, Itay Dalmedigos, Dor Muhlgay, Amnon Shashua, Kevin Leyton-Brown, and Yoav Shoham. 2023. In-Context Retrieval-Augmented Language Models. *Transactions of the Association for Computational Linguistics* (2023). <https://arxiv.org/abs/2302.00083>
- [21] Stephen E. Robertson, Steve Walker, Micheline Hancock-Beaulieu, Aaron Gull, and Marianna Lau. 1992. Okapi at TREC. In *Proceedings of TREC*. 21–30.
- [22] Zhihong Shao, Yeyun Gong, Yelong Shen, Minlie Huang, Nan Duan, and Weizhu Chen. 2023. Enhancing Retrieval-Augmented Large Language Models with Iterative Retrieval-Generation Synergy. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. 9248–9274.
- [23] Anna Shtok, Oren Kurland, David Carmel, Fiana Raiber, and Gad Markovits. 2012. Predicting Query Performance by Query-Drift Estimation. *ACM Trans. Inf. Syst.* 30, 2 (2012), 11:1–11:35.
- [24] William Webber, Alistair Moffat, and Justin Zobel. 2010. A similarity measure for indefinite rankings. *ACM Trans. Inf. Syst.* 28, 4, Article 20 (2010).
- [25] Shicheng Xu, Liang Pang, Huawei Shen, and Xueqi Cheng. 2024. A Theory for Token-Level Harmonization in Retrieval-Augmented Generation. *arXiv:2406.00944* [cs.CL] <https://arxiv.org/abs/2406.00944>
- [26] Chanwoong Yoon, Gangwoo Kim, Byeongguk Jeon, Sungdong Kim, Yohan Jo, and Jaewoo Kang. 2024. Ask Optimal Questions: Aligning Large Language Models with Retriever's Preference in Conversational Search. *arXiv preprint arXiv:2402.11827* (2024).
- [27] Wenhao Yu, Dan Iter, Shuohang Wang, Yichong Xu, Mingxuan Ju, Sanyal, Chenguang Zhu, Michael Zeng, and Meng Jiang. 2023. Generate rather than Retrieve: Large Language Models are Strong Context Generators. In *International Conference on Learning Representations*.
- [28] Oleg Zendel, Anna Shtok, Fiana Raiber, Oren Kurland, and J Shane Culpepper. 2019. Information needs, queries, and query performance prediction. In *Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 395–404.
- [29] Yun Zhou and W Bruce Croft. 2007. Query performance prediction in web search environments. In *Proceedings of the 30th annual international ACM SIGIR conference on Research and development in information retrieval*. 543–550.